

Cilium Flow

Real-time Network Observability & Intelligence Platform for Kubernetes

Cilium Flow Team

March 2026

Contents

Executive Summary	2
Platform Capabilities	3
Live Observability	3
Intelligence Engines	3
Operations Engines	5
Architecture	6
Quality Assurance	6
Deployment	7
Requirements	7
Installation	7
First Run	7
Roadmap	7
Contact	7

Executive Summary

Cilium Flow is a single-binary terminal application that transforms Kubernetes network operations from reactive firefighting into proactive, intelligent management.

Built in Rust for maximum performance, it combines live eBPF datapath monitoring with eight autonomous intelligence engines that detect problems, generate policies, simulate changes, inject faults, manage canary rollouts, and orchestrate multi-cluster environments — all from one console.

Key Metrics:

Metric	Value
Tests passing	969 (0 failures, 0 warnings)
Lines of Rust	11,400+
Binary size	13 MB
Modules	13
Interactive tabs	13
CPU overhead	< 1%
Memory footprint	50–100 MB

Platform Capabilities

Live Observability

Cilium Flow streams Hubble flows in real time with verdict-based coloring. Every packet can be explained with a single keypress, showing why it was forwarded or dropped, which policy matched, and what action to take.

Five observability tabs provide complete visibility:

- **Flows** — live packet stream with per-packet explanation
- **Connections** — enriched conntrack data with pod metadata
- **Endpoints** — auto-discovered Cilium endpoints
- **Policies** — active CiliumNetworkPolicy visualization
- **Metrics** — eBPF datapath aggregate statistics

Intelligence Engines

Self-Healing (Healer)

Analyzes eBPF drop counters to detect network problems — policy denials, unreachable backends, MTU issues, DNS failures — and proposes targeted fixes. Operates in dry-run mode by default.

AutoPolicy

Learns communication patterns from conntrack data and generates CiliumNetworkPolicy YAML. Each recommendation includes a 7-feature ML confidence score:

Confidence	Meaning	Action
90–100%	High confidence	Safe for production
75–89%	Moderate	Test in staging
60–74%	Low	Audit mode
< 60%	Insufficient data	Manual review

Policies can be applied, rolled back, or batch-managed directly from the TUI.

Root Cause Analysis

Correlates dropped packets with active policies to identify the root cause of connectivity failures. Offers one-click fixes that generate and apply corrective CiliumNetworkPolicy resources.

Dry-Run Simulator

Test policy changes before production. Seven built-in scenarios (add policy, default deny, block traffic, etc.) with full impact analysis: flows affected, services impacted, and risk level scoring.

Time-Travel Debugging

Record network flows to disk and replay them with VCR-style controls — play, pause, step forward/backward, jump to events, adjustable playback speed. Enables post-incident analysis without reproducing the failure.

Operations Engines

Chaos Engineering

Controlled fault injection using kernel tc-netem rules, applied and removed via kubectl exec. Seven pre-built experiments:

Experiment	Parameters	Severity
Network Partition	20% packet drop	Medium
Latency Spike	500ms +/- 50ms	Medium
DNS Outage	30% DNS failure	High
Connection Reset	15% connection kill	Medium
Bandwidth Limit	10 Mbps cap	Low
Packet Corruption	5% corruption	High
Total Partition	100% drop	Critical

Safety features: configurable limits (max 50% drop, max 5s latency), circuit breaker for emergency shutdown, automatic cleanup after 5 minutes, confirmation required for all experiments.

Canary Deployments

Progressive traffic shifting without sidecars, using Cilium's native L7 load balancing:

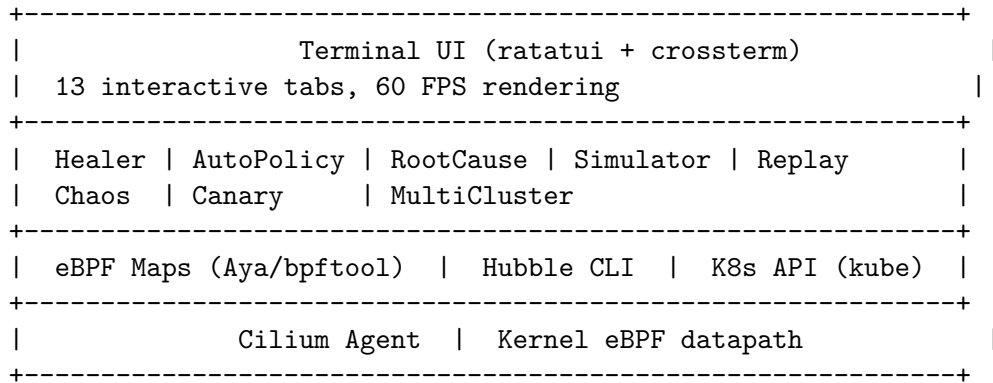
- Start at 10% canary traffic, increment by 10% per step
- Auto-promote when success rate exceeds 99%
- Auto-rollback when success rate drops below 90%
- Real-time visualization: traffic split bars, health gauges, latency comparison

Multi-Cluster Autopilot

Cross-cluster orchestration for multi-cloud and hybrid deployments:

- Register clusters with provider, region, and API endpoint metadata
- Health-check all clusters via kubectl (node readiness, pod counts)
- Topology visualization with connected pairs and region grouping
- Policy synchronization across clusters
- Intelligent workload placement recommendations

Architecture



Design principles:

- All async I/O uses `tokio::process::Command` (no blocking on the event loop)
- Dual data path: `EnrichedMapReader` for real eBPF data, `MockMapReader` fallback
- Confirmation workflow for destructive operations
- Engine independence: operations engines (Chaos, Canary, MultiCluster) have no eBPF dependency

Quality Assurance

Check	Result
Unit tests	369 passing
Integration tests	600 passing
Total tests	969 passing, 0 failures
Compiler warnings	0
Clippy lints	0
Unsafe code	0 blocks
Code review passes	4 (all issues resolved)

The test suite covers core logic, parsers, engine state machines, cross-module interaction, policy generation, simulation impact analysis, and tab navigation.

Deployment

Requirements

Component	Minimum
Kubernetes	1.25+
Cilium	1.14+
Rust (build)	1.75+

Installation

```
git clone https://github.com/ssahani/cilium-flow.git
cd cilium-flow
cargo build --release
sudo cp target/release/cilium-tui /usr/local/bin/
```

First Run

```
cilium-tui                # Full auto-bootstrap
cilium-tui --skip-bootstrap # If Cilium/Hubble already running
cilium-tui --auto-install  # Non-interactive Cilium install
```

The tool auto-detects the cluster, verifies Cilium, enables Hubble relay, starts port-forwarding, and launches the TUI.

Roadmap

Phase	Focus
Current	All 8 engines wired to TUI, 969 tests, production-ready core
Next	Prometheus exporter, Grafana dashboards, REST API
Future	Web UI, plugin system (WASM), deep learning anomaly models

Contact

- Repository: github.com/ssahani/cilium-flow
- Issues: GitHub Issues